GALAXY IMAGE CLASSIFICATION

Convolutional Neural Networks, Transfer Learning and Grad-CAM

Detailed Technical Report

Based on the submitted Jupyter notebook and presentation

Project authors: Matteo Monacis and Francesco Renna
Academic year 2024/2025

Scope of this report

The report explains the complete implemented pipeline, interprets the experimental results, corrects the Galaxy10 class taxonomy, and identifies limitations that affect reproducibility and scientific interpretation.

Abstract

This project investigates automatic galaxy morphology classification from colour astronomical images. A 50% random sample of the Galaxy10 DECaLS dataset was resized to 128 x 128 pixels, encoded for ten-class classification, divided into stratified training, validation and test partitions, normalized and standardized, and partially rebalanced by augmenting the rarest class. Five convolutional models were then compared: a custom CNN trained from scratch and four ImageNet-pretrained feature extractors (VGG16, DenseNet201, ResNet50 and MobileNetV2). All pretrained backbones were frozen, so the experiment represents fixed-feature transfer learning rather than fine-tuning. DenseNet201 achieved the highest saved test accuracy (57.48%) and weighted F1 score (0.56), with ResNet50 close behind but showing pronounced overfitting. The custom model underfit severely and reached only 31.25% test accuracy. Grad-CAM visualizations were produced for VGG16, DenseNet201 and MobileNetV2, although a preprocessing inconsistency makes those visual explanations exploratory rather than fully controlled. The analysis also finds a critical class-name mapping error in the notebook: the official Galaxy10 DECaLS classes 6, 7 and 9 were mislabeled, and the dataset contains no “Irregular” class. This report therefore presents all class-level results using the corrected taxonomy while documenting the labels used in the original implementation.

> **Primary conclusion**
>
> The experiment supports the value of transfer learning for this small, imbalanced image-classification task. DenseNet201 provides the strongest overall balance, while the results also show that correct label semantics, architecture-specific preprocessing, controlled random seeds and consistent model selection are essential for a defensible comparison.

Contents

1. Scientific and Project Context

1.1 Why galaxy morphology matters

A galaxy's visible morphology is connected to its structure and evolutionary history. Broad categories such as smooth galaxies, spiral galaxies, interacting systems and edge-on disks reflect different distributions of stars, dust, gas and dynamical components. Large astronomical surveys generate more images than can be classified consistently by manual inspection, which motivates automated computer-vision methods.

The task is difficult because the classes are not separated by a single simple visual feature. Orientation changes the apparent shape; spiral arms can be faint; foreground stars and background objects introduce clutter; and several labels form a continuum rather than sharply separated categories. Consequently, the model must learn both local features - for example arm segments, bars and tidal distortions - and global structure such as elongation, symmetry and orientation.

1.2 Project goals

- **Build a complete classification pipeline.** Load, preprocess, train and evaluate models on Galaxy10 DECaLS images.
- **Compare learning strategies.** Contrast a CNN trained from scratch with pretrained CNN backbones used through transfer learning.
- **Manage computational constraints.** Reduce image resolution and use only half of the available dataset to make training feasible on the selected notebook environment.
- **Address class imbalance.** Increase the representation of the rarest class by generating transformed training images.
- **Evaluate beyond accuracy.** Use class-level precision, recall and F1 scores, together with confusion matrices where the notebook preserved them.
- **Inspect model attention.** Use Grad-CAM to visualize image regions that influence selected predictions.

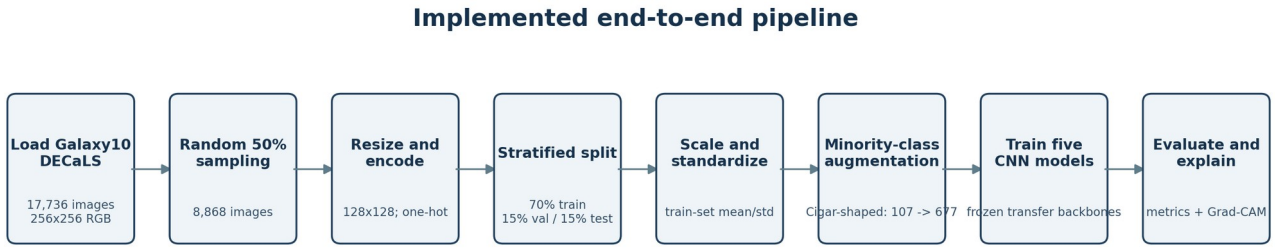

Figure 1. End-to-end pipeline reconstructed from the notebook implementation.

2. Dataset and Class Taxonomy

2.1 Galaxy10 DECaLS

The project loads Galaxy10 DECaLS through astroNN. The complete dataset contains 17,736 RGB images of shape 256 x 256 x 3. The colour channels originate from astronomical survey bands and are provided as display-ready three-channel images. Labels are integer class indices from 0 to 9 and are converted to ten-dimensional one-hot vectors before training.

Index	Official Galaxy10 DECaLS class	Images	Share
0	Disturbed	1,081	6.09%
1	Merging	1,853	10.45%
2	Round Smooth	2,645	14.91%

Index	Official Galaxy10 DECaLS class	Images	Share
3	In-between Round Smooth	2,027	11.43%
4	Cigar-shaped Smooth	334	1.88%
5	Barred Spiral	2,043	11.52%
6	Unbarred Tight Spiral	1,829	10.31%
7	Unbarred Loose Spiral	2,628	14.82%
8	Edge-on without Bulge	1,423	8.02%
9	Edge-on with Bulge	1,873	10.56%

Table 1. Official Galaxy10 DECaLS taxonomy and class distribution.

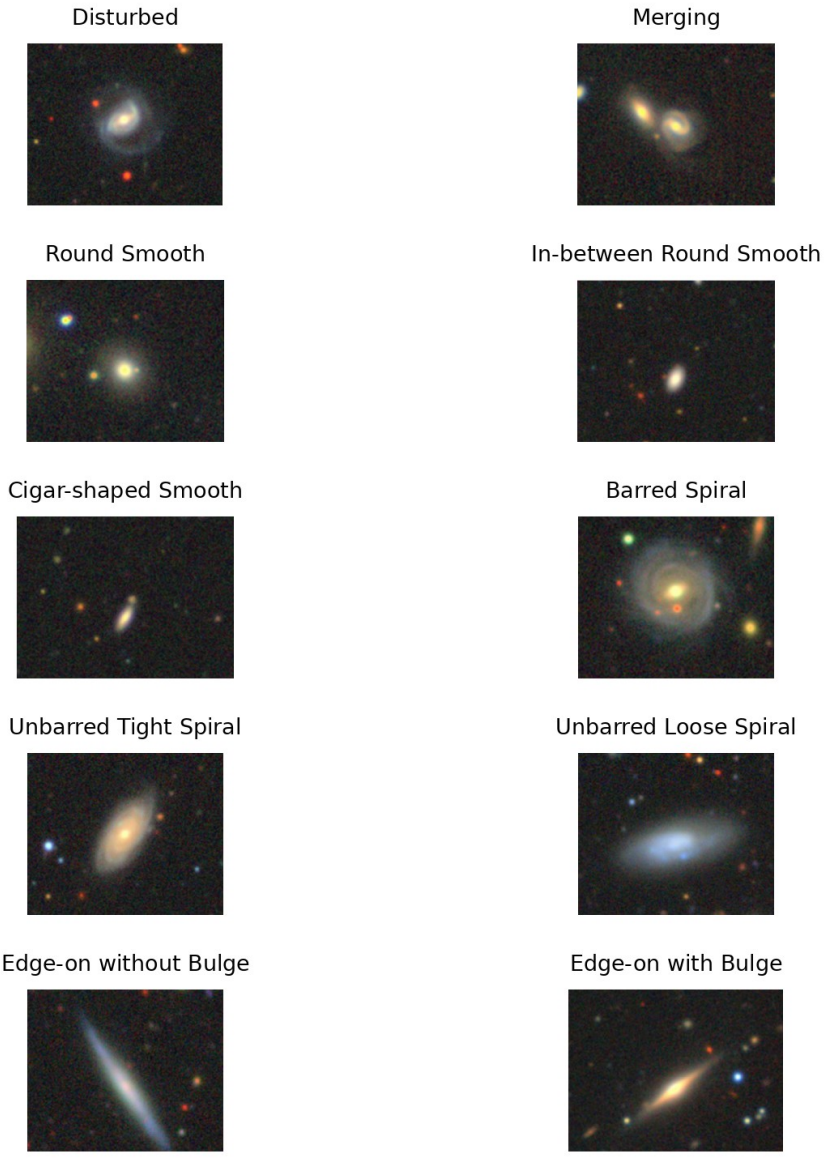

Figure 2. Representative class examples extracted from the project presentation and relabeled according to the official Galaxy10 DECaLS taxonomy.

2.2 Critical label-mapping correction

Important implementation issue

The notebook class_names list does not match the official Galaxy10 DECaLS class order. Index 6 should be “Unbarred Tight Spiral,” index 7 should be “Unbarred Loose Spiral,” and index 9 should be “Edge-on Galaxies with Bulge.” The notebook instead uses “Unbarred Spiral,” “Edge-on w/ Bulge,” and “Irregular.” Galaxy10 DECaLS has no Irregular class. Predictions and numeric metrics remain valid at the index level, but their textual interpretation is wrong unless the labels are corrected.

Class index	Notebook label	Correct label	Issue
6	Unbarred Spiral	Unbarred Tight Spiral	Incomplete name
7	Edge-on w/ Bulge	Unbarred Loose Spiral	Wrong morphology
9	Irregular	Edge-on with Bulge	Non-existent dataset class

Table 2. Label discrepancies found between the notebook and the official dataset taxonomy.

2.3 Dataset reduction

The code retains a random 50% subset before any train/validation/test split: **sample_frac = 0.50 -> 17,736 x 0.50 = 8,868 images**

This decision reduces memory use and training time. However, the random sampling call has no explicit seed and is not stratified, so the exact retained images and class counts can change whenever the notebook is rerun. The subsequent data split is stratified, which preserves the class proportions of whichever 50% sample happened to be selected.

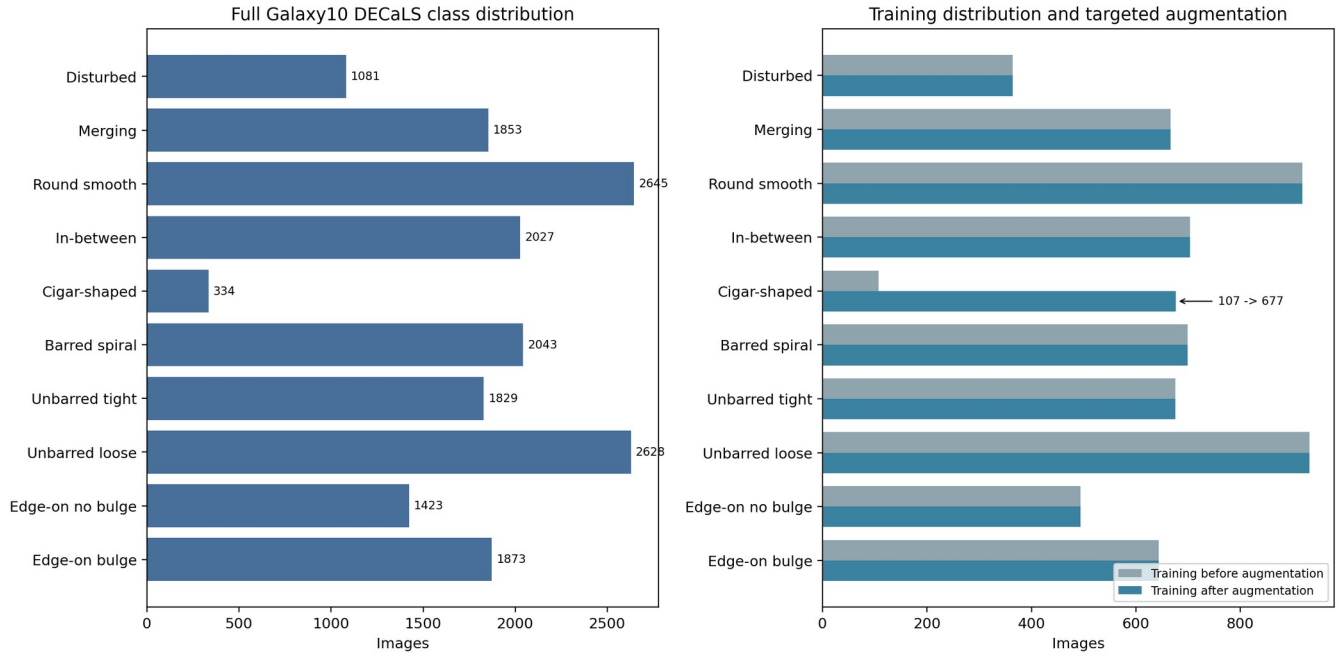

Figure 3. Full-dataset class imbalance and the targeted augmentation applied to the sampled training set. The sampled training counts were reconstructed from the saved split supports and verified against the notebook distribution plot.

3. Data Preparation and Preprocessing

3.1 Loading and one-hot encoding

astroNN returns an image array and an integer label array. The labels are transformed with to_categorical(labels, num_classes=10). For a sample belonging to class 2, the target becomes [0, 0, 1, 0, ..., 0]. This representation is compatible with the ten-unit softmax output and categorical cross-entropy loss used by every model.

3.2 Resizing

Every image is resized from 256 x 256 to 128 x 128 pixels with tf.image.resize. The operation reduces each image from 196,608 scalar values to 49,152, a fourfold reduction in spatial data. This materially lowers activation-memory use and computation. The presentation describes this as cropping, but the notebook performs resizing; these operations are not equivalent. Resizing preserves the full field of view while reducing detail, whereas cropping would remove outer regions.

3.3 Stratified partitioning

The sampled data are split in two stages using random_state=42 and stratification. First, 70% becomes training data and 30% becomes a temporary set. The temporary set is then divided equally into validation and test partitions. The resulting sizes are 6,207 training images, 1,330 validation images and 1,331 test images.

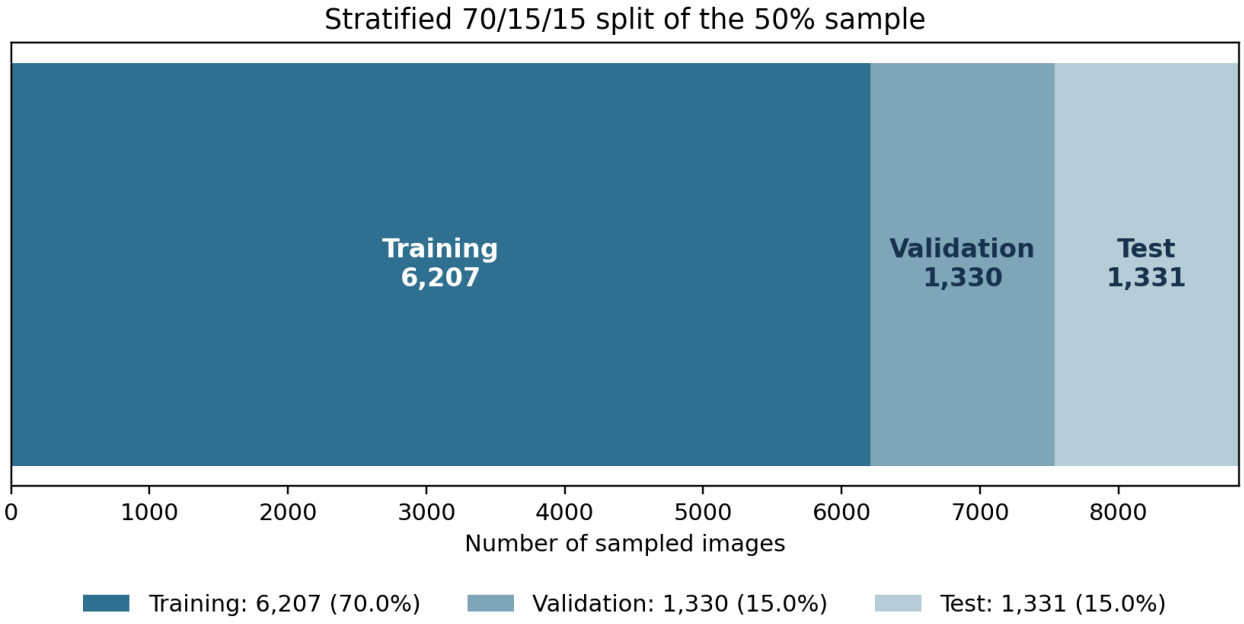

Figure 4. Sizes of the stratified training, validation and test partitions.

Stratification is important because the rare Cigar-shaped Smooth class represents less than 2% of the original dataset. Without stratification, a small validation or test split could contain too few examples to produce stable class-level metrics.

3.4 Normalization and standardization

The notebook applies two successive transformations:

1. Normalization: convert to float32 and divide every pixel by 255, producing values approximately in [0, 1].

2. Standardization: compute one mean and standard deviation per colour channel on the normalized training set, then transform every partition as x_std = (x - mean_train) / std_train.

Computing mean and standard deviation only from the training partition is correct: it prevents information from the validation and test sets from influencing preprocessing. The same training-derived statistics are then applied to all partitions.

> **Transfer-learning caveat**
>
> The same standardized tensors are fed to every pretrained backbone, but Keras VGG16, ResNet50, DenseNet201 and MobileNetV2 each provide an architecture-specific preprocess_input function. Not using the preprocessing associated with the original ImageNet training distribution can reduce feature-transfer quality and makes the comparison less representative of each architecture's intended operating conditions.

3.5 Targeted minority-class augmentation

Class counts are calculated only in the training set. The least represented class is Cigar-shaped Smooth, with 107 sampled training images. The target is the integer mean of the other nine training-class counts: 677 images. The code generates 570 synthetic images and appends them to the original training set, raising the training-set size from 6,207 to 6,777.

Transformation	Notebook setting
Rotation	Up to 40 degrees
Horizontal shift	Up to 20% of width
Vertical shift	Up to 20% of height
Shear	Up to 20%
Zoom	Up to 20%
Horizontal flip	Enabled
Fill mode	Nearest-neighbour

Table 3. ImageDataGenerator transformations used to synthesize minority-class examples.

The augmentation is generated once before training and then stored in X_train_balanced. The ImageDataGenerator instances used by model.fit do not apply additional transformations during each epoch. Therefore, this is offline augmentation of one class, not online augmentation of the complete training set. The procedure reduces the most extreme imbalance but does not make all ten classes equal.

3.6 Data generators and batch construction

Three Keras flows are built with batch size 32. The training flow shuffles samples, whereas the validation and test flows set shuffle=False. Disabling test shuffling is essential because predicted rows must remain aligned with y_test when the classification report and confusion matrix are computed. With 6,777 balanced training examples, each epoch contains ceil(6,777 / 32) = 212 training steps, matching the notebook logs.

4. Experimental Design and Evaluation Protocol

4.1 Common training setup

- **Input tensor.** 128 x 128 x 3 standardized images.
- **Output.** Ten softmax probabilities whose sum is one.
- **Loss.** Categorical cross-entropy: L = -sum_i y_i log(p_i).
- **Optimizer.** Adam with its default 0.001 learning rate, except the custom model where the same rate is written explicitly.
- **Maximum training duration.** 15 epochs.
- **Batch size.** 32 images.
- **Backbone policy.** Every pretrained convolutional layer is frozen; only the newly attached classifier is trained.
- **Model checkpointing.** Save the model with the lowest validation loss.
- **Early stopping.** Stop after five epochs without lower validation loss and restore the best validation-loss weights.

4.2 What “transfer learning” means here

Transfer learning can be performed in two stages: fixed feature extraction and fine-tuning. This project uses only fixed feature extraction. ImageNet-trained convolutional filters remain unchanged, and a new ten-class output head learns from the galaxy images. This strategy is computationally inexpensive and helps when the task-specific dataset is limited, but it prevents deeper filters from adapting to astronomical textures and morphology.

4.3 Evaluation metrics

Metric	Definition	Interpretation
Accuracy	Correct predictions / all predictions	Overall correctness; sensitive to class imbalance
Precision for class c	TP_c / (TP_c + FP_c)	How reliable predictions of class c are
Recall for class c	TP_c / (TP_c + FN_c)	How many true class-c images are recovered
F1 for class c	2PR / (P + R)	Balances precision and recall
Macro average	Unweighted mean across classes	Treats rare and common classes equally
Weighted average	Mean weighted by class support	Reflects the observed test distribution
Confusion matrix	True class by predicted class counts	Shows specific error pathways

Table 4. Metrics used in the project.

Macro F1 is particularly important here because overall accuracy can remain moderate even when rare classes are almost never recognized. For example, VGG16 reaches 54.77% test accuracy but only 4% recall on Cigar-shaped Smooth images.

5. Model Architectures

5.1 Custom CNN trained from scratch

The custom model contains four valid 3 x 3 convolutions with 16, 32, 64 and 128 filters. Each convolution is followed by 2 x 2 max pooling with stride 1. Because the pooling stride is one rather than two, the feature maps shrink slowly: the final map remains approximately 116 x 116 x 128. Global average pooling compresses this map to 128 values, followed by 50% dropout, a 64-unit tanh dense layer and a ten-unit softmax output.

- **Strength.** Small parameter count and complete control over the architecture.
- **Weakness.** All visual features must be learned from only 6,777 training examples, including synthetic images.
- **Design concern.** tanh can saturate, and stride-1 pooling preserves very large activation maps without giving the usual rapid spatial reduction of CNNs.
- **Presentation inconsistency.** The slide text describes ReLU activations, while the actual code uses tanh in every convolution and hidden dense layer.

5.2 VGG16

VGG16 is a sequential deep CNN built from stacks of 3 x 3 convolutions and max-pooling operations. The ImageNet classification top is removed. At 128 x 128 input size, the final convolutional feature map is 4 x 4 x 512. Global average pooling creates a 512-dimensional feature vector, and a ten-unit softmax layer introduces only 5,130 trainable parameters.

5.3 DenseNet201

DenseNet connects each layer to later layers within a dense block, encouraging feature reuse and improving gradient flow. The frozen backbone produces a 4 x 4 x 1,920 map. Global average pooling and the ten-class dense output introduce 19,210 trainable parameters. This is the highest-dimensional pooled representation among the GAP-based transfer models and produced the best overall test result.

5.4 ResNet50

ResNet uses residual shortcut connections that allow very deep networks to learn transformations relative to an identity path. The project removes the original top but applies Flatten rather than global average pooling to the 4 x 4 x 2,048 backbone output. The resulting 32,768-element vector feeds the softmax layer, producing 327,690 trainable parameters. This head is far larger than the heads used for the other frozen backbones and is a plausible contributor to the observed overfitting.

5.5 MobileNetV2

MobileNetV2 is designed for efficient inference through depthwise separable convolutions, inverted residual blocks and linear bottlenecks. The 4 x 4 x 1,280 output is globally averaged, then connected to ten softmax units. The classifier contains approximately 12,810 trainable parameters, making this the most compact pretrained architecture in the experiment.

Model	Pretraining	Backbone output	Classification head	Architecture params	Trainable params
Custom CNN	None	Final conv ~116 x 116 x 128	GAP -> Dropout -> Dense 64 -> Dense 10	106,346	106,346
VGG16	ImageNet	4 x 4 x 512	GAP -> Dense 10	14,719,818	5,130
DenseNet201	ImageNet	4 x 4 x 1,920	GAP -> Dense 10	18,341,194	19,210
ResNet50	ImageNet	4 x 4 x 2,048	Flatten -> Dense 10	23,915,402	327,690
MobileNetV2	ImageNet	4 x 4 x 1,280	GAP -> Dense 10	~2,270,794	12,810

Table 5. Architecture comparison. Optimizer-state variables shown by some Keras summaries are excluded from architecture parameter counts.

6. Experimental Results

6.1 Overall comparison

Model	Final train acc.	Final val acc.	Test acc.	Macro F1	Weighted F1	Epochs run
Custom CNN	0.3346	0.3098	0.3125	0.22	0.27	15/15
VGG16	0.5979	0.5271	0.5477	0.48	0.53	15/15
DenseNet201	0.6921	0.5647	0.5748	0.52	0.56	15/15
ResNet50	0.8577	0.5459	0.5665	0.52	0.55	8/15
MobileNetV2	0.6439	0.4474	0.4884	0.45	0.47	12/15

Table 6. Saved notebook performance. Final train/validation values come from the last logged epoch, while test evaluation uses weights restored from the lowest-validation-loss epoch.

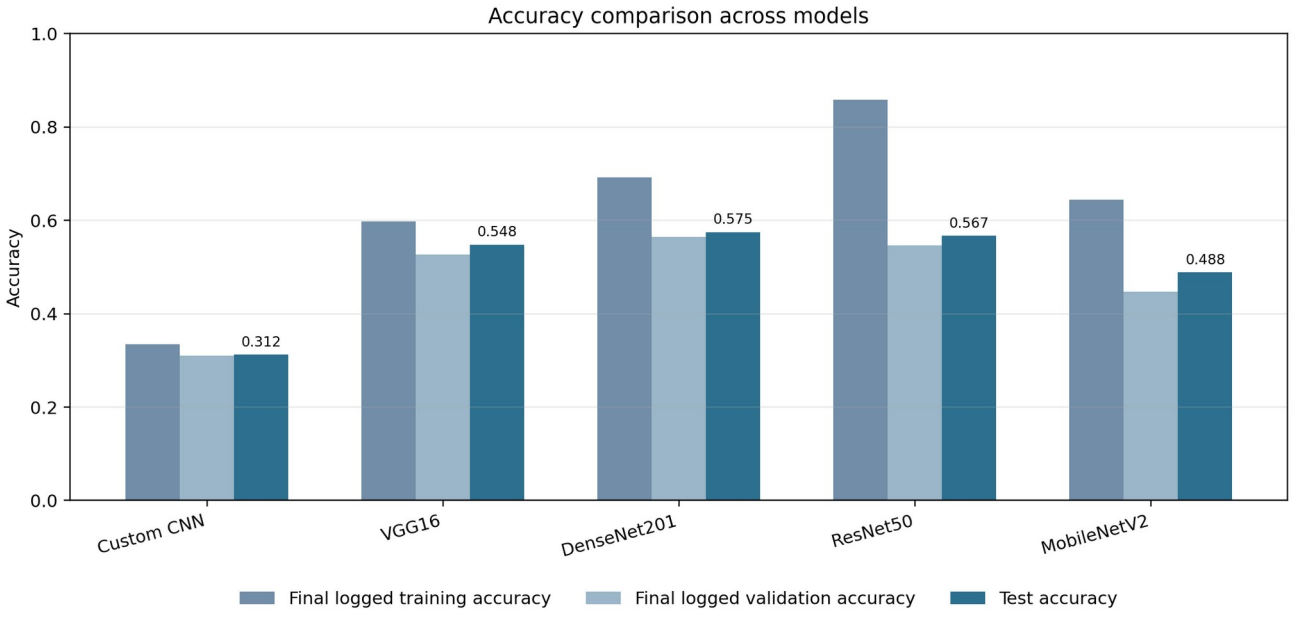

Figure 5. Training, validation and test accuracies reported by the notebook.

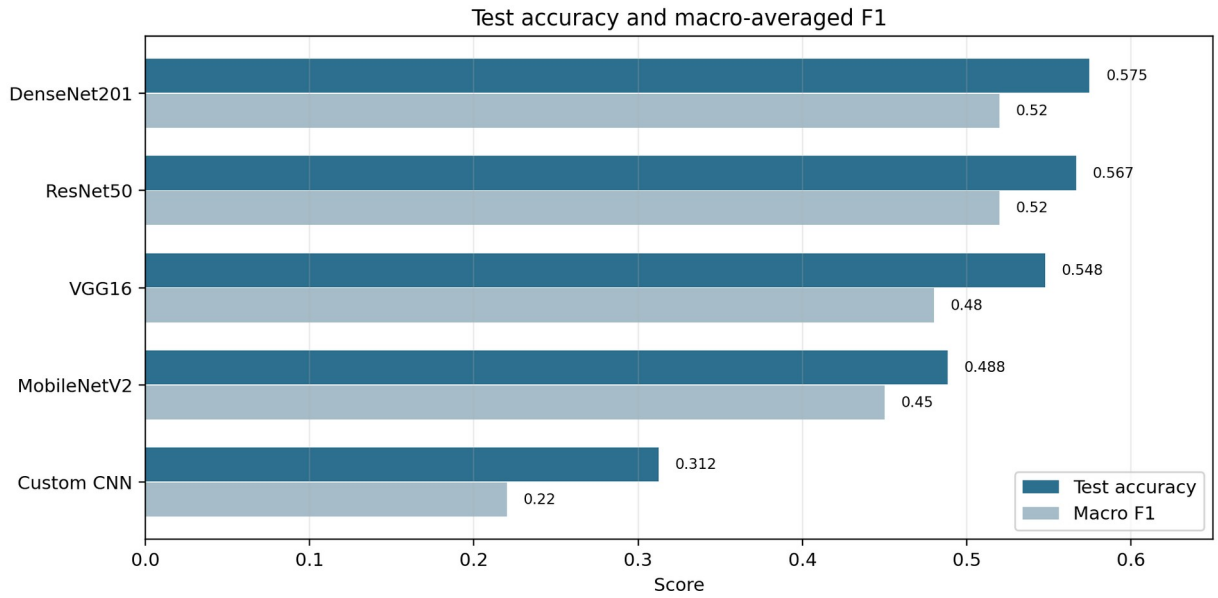

Figure 6. Test accuracy and macro F1. DenseNet201 has the highest test accuracy and shares the best macro F1 with ResNet50.

Correction to the presentation comparison table

The presentation reports a macro F1 score of 0.52 for the Custom CNN. The notebook classification report gives macro F1 = 0.22 and weighted F1 = 0.27. The report uses the notebook output, which is consistent with the near-zero performance on several classes.

DenseNet201 is the strongest model by test accuracy (57.48%) and weighted F1 (0.56). ResNet50 is close in accuracy (56.65%) and macro F1 (0.52), but its learning curve shows much stronger overfitting. VGG16 is slightly weaker overall but performs very well on the two edge-on categories. MobileNetV2 provides a lower-cost model but loses about 8.6 percentage points of test accuracy relative to DenseNet201. The custom model substantially underfits.

6.2 Learning dynamics

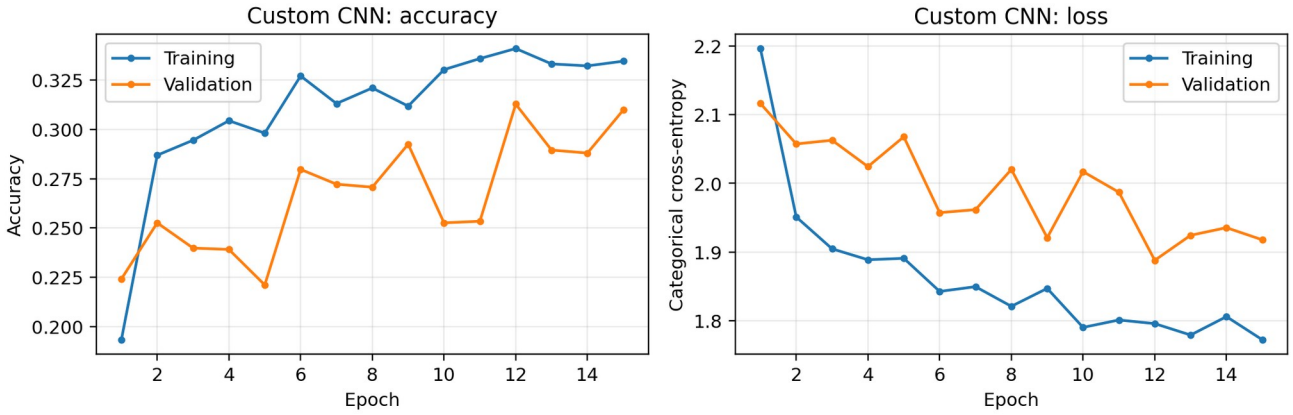

Figure 7. Custom CNN learning curves. Both training and validation accuracy remain low, indicating underfitting rather than classical overfitting.

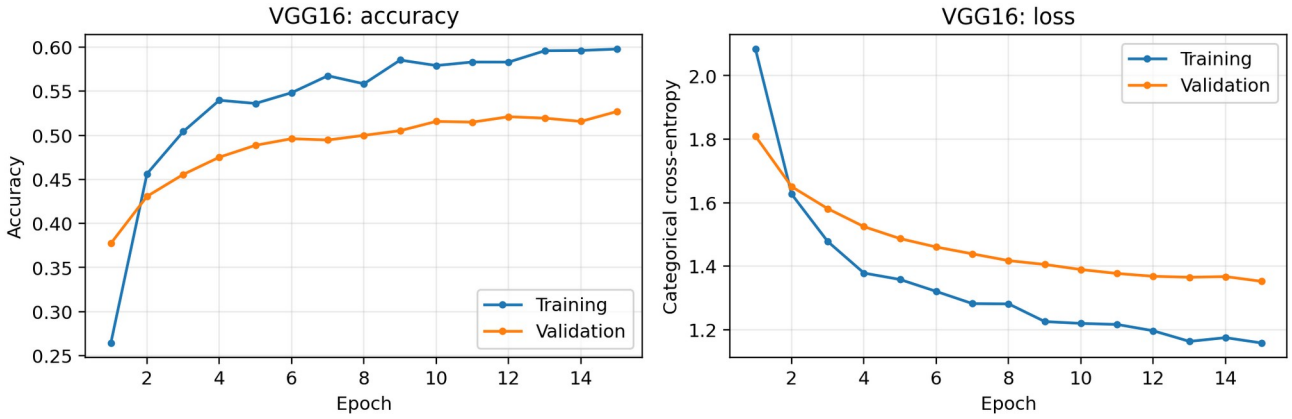

Figure 8. VGG16 learning curves. Accuracy improves gradually and the validation loss declines throughout training.

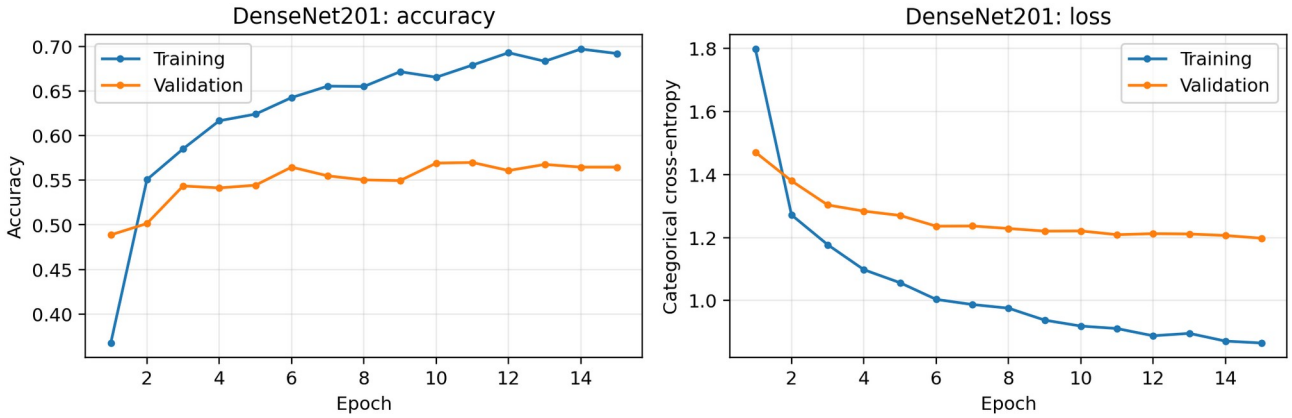

Figure 9. DenseNet201 learning curves. Validation performance stabilizes while training accuracy continues to improve, producing a moderate generalization gap.

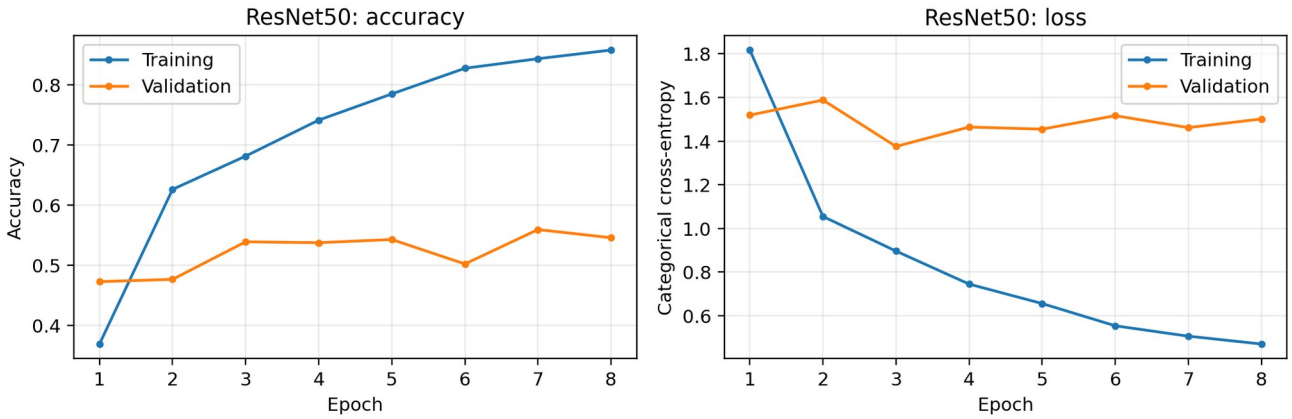

Figure 10. ResNet50 learning curves. Training accuracy rises rapidly, while validation loss is best near epoch 3 and then deteriorates; early stopping ends training after epoch 8 and restores the best-loss weights.

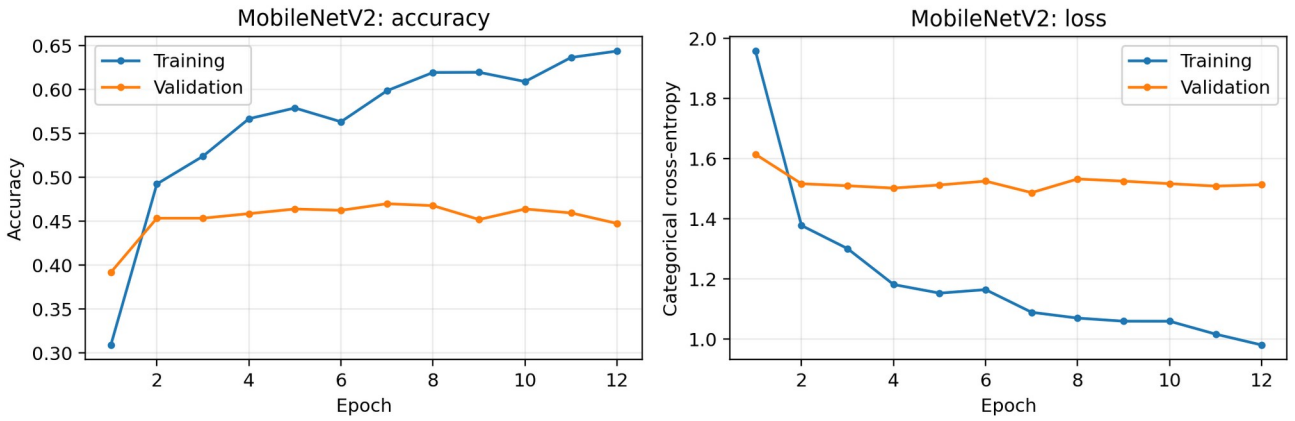

Figure 11. MobileNetV2 learning curves. Validation accuracy plateaus below 0.47 and the best validation loss occurs around epoch 7.

> **How to read the reported ResNet50 and MobileNetV2 values**
>
> For ResNet50, the final log shows training accuracy 0.8577 and validation accuracy 0.5459, but restore_best_weights=True returns the model to the epoch with the lowest validation loss (epoch 3). MobileNetV2 similarly restores approximately epoch 7. The test results therefore do not correspond to the final-epoch training accuracies printed in the presentation. A rigorous report should state both the selected epoch and the selected model's metrics.

6.3 Class-level performance

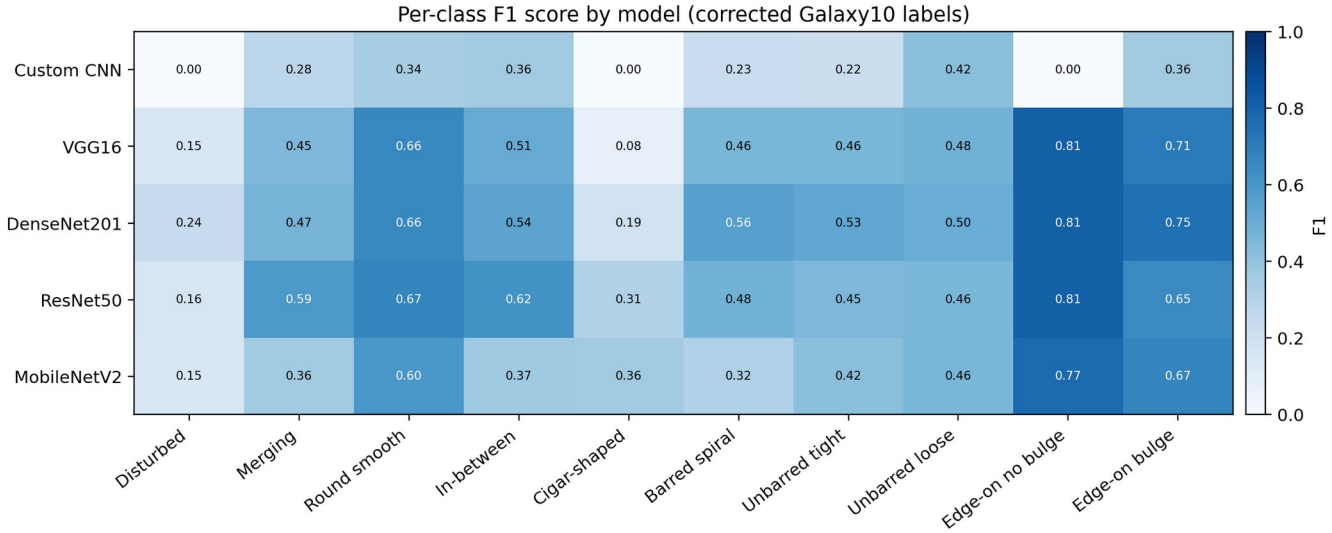

Figure 12. Per-class F1 scores for all models using the corrected official class names.

The heatmap reveals four recurring patterns:

- **Disturbed galaxies are difficult.** The best F1 is only 0.24 (DenseNet201), suggesting that irregular tidal features are either weak, diverse or confused with mergers and spiral structure.
- **Cigar-shaped Smooth is unstable.** Despite augmentation, the class has only 23 test images. MobileNetV2 reaches the highest F1 (0.36), while VGG16 recovers only one of the 23 images.
- **Round Smooth is comparatively easy.** VGG16, DenseNet201 and ResNet50 all reach F1 values around 0.66-0.67, driven by high recall.
- **Edge-on categories are distinctive.** Edge-on without Bulge reaches F1 = 0.81 for VGG16, DenseNet201 and ResNet50. Edge-on with Bulge is also strong, especially for DenseNet201 at F1 = 0.75.
- **Spiral subclasses remain confusable.** Barred, unbarred tight and unbarred loose structures have only moderate F1 values, consistent with subtle arm and bar visibility at 128 x 128 resolution.

6.4 Confusion-matrix analysis

The saved notebook contains exact confusion matrices only for VGG16 and DenseNet201. The original plots use the incorrect notebook labels; the following matrices preserve the exact counts but use the official Galaxy10 DECaLS taxonomy. Exact matrices for the Custom CNN, ResNet50 and MobileNetV2 cannot be reconstructed uniquely from rounded classification reports alone.

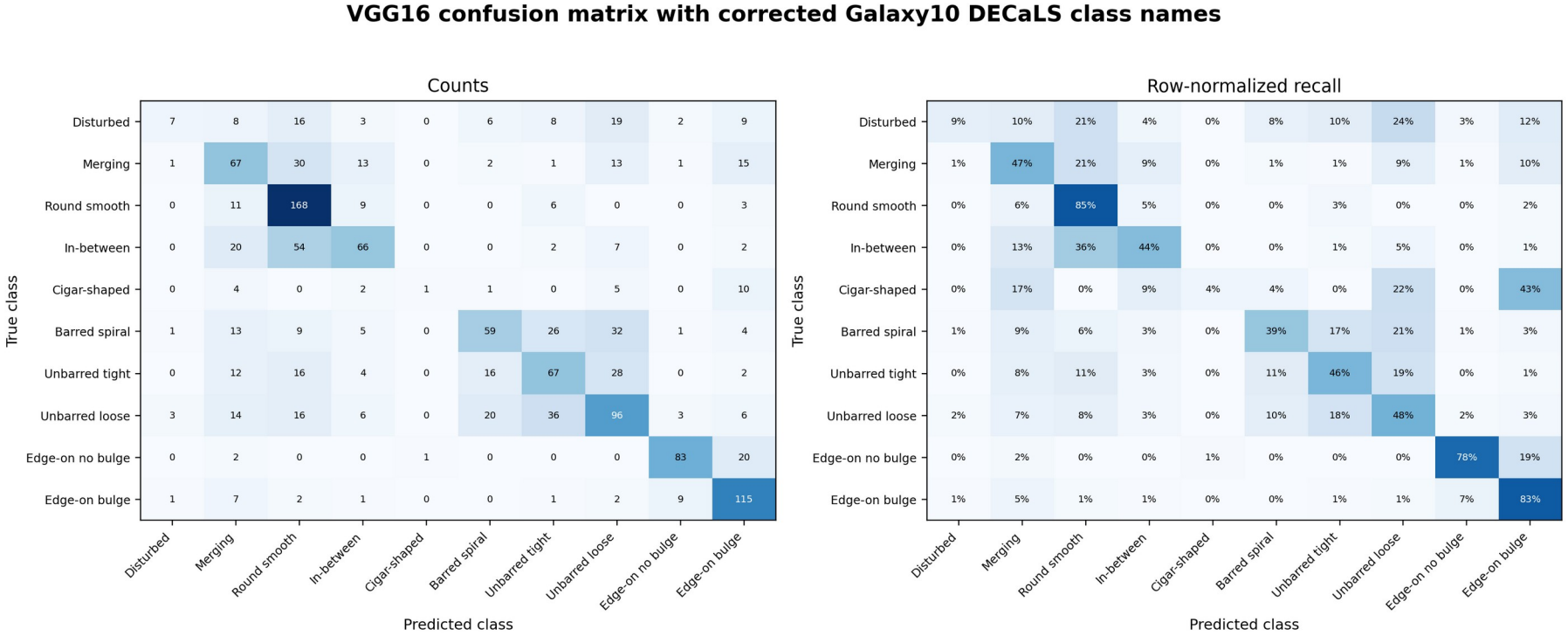

Figure 13. VGG16 confusion matrix: raw counts and row-normalized percentages.

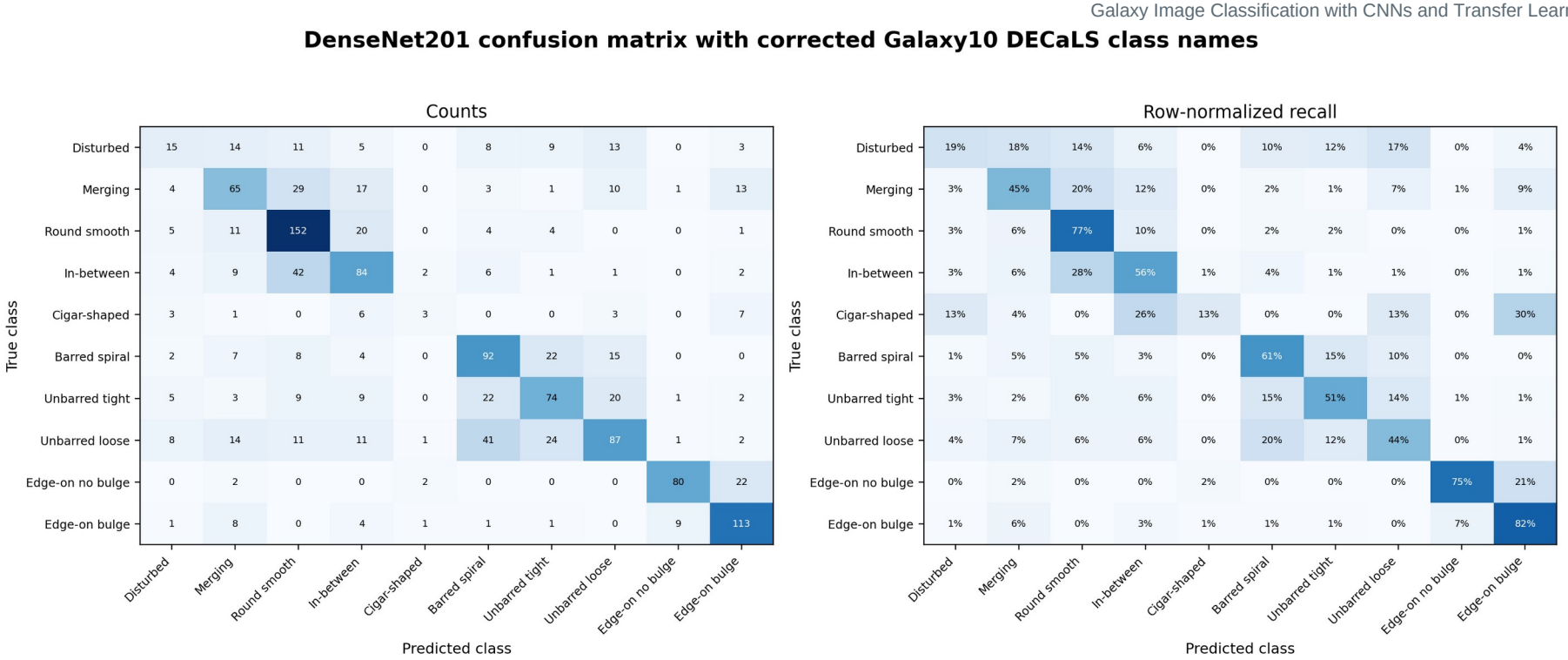

Figure 14. DenseNet201 confusion matrix: raw counts and row-normalized percentages.

6.5 What the confusion matrices show

- **VGG16 collapses several categories toward Round Smooth.** For example, 54 of 151 In-between Round Smooth images are predicted as Round Smooth. This reflects the visual continuum between the two classes.
- **DenseNet201 improves In-between Round Smooth recognition.** It correctly classifies 84 images compared with VGG16's 66, while reducing the flow into Round Smooth from 54 to 42.
- **Spiral and loose-spiral confusion is substantial.** DenseNet201 predicts 41 of 200 Unbarred Loose Spiral images as Barred Spiral and 24 as Unbarred Tight Spiral.
- **The rare Cigar-shaped class remains poorly estimated.** DenseNet201 correctly identifies 3 of 23, while VGG16 identifies 1 of 23; many are assigned to Edge-on with Bulge or Unbarred Loose Spiral.
- **Edge-on without Bulge is highly recognizable but can be confused with Edge-on with Bulge.** VGG16 assigns 20 of 106 to the bulged class; DenseNet201 assigns 22.

7. Model Explainability with Grad-CAM

7.1 Principle

Gradient-weighted Class Activation Mapping (Grad-CAM) estimates which locations in the last convolutional feature maps support a selected class. For class c, the gradients of the class score with respect to each feature map are spatially averaged to obtain a weight alpha_k^c. The weighted feature maps are summed and passed through ReLU, producing a coarse positive-evidence heatmap: L_Grad-CAM^c = ReLU(sum_k alpha_k^c A^k). The heatmap is resized to the input resolution and overlaid on the display image.

The notebook applies Grad-CAM to the final convolutional layers block5_conv3 for VGG16, conv5_block32_concat for DenseNet201, and Conv_1 for MobileNetV2. Three random test images are selected separately for each model.

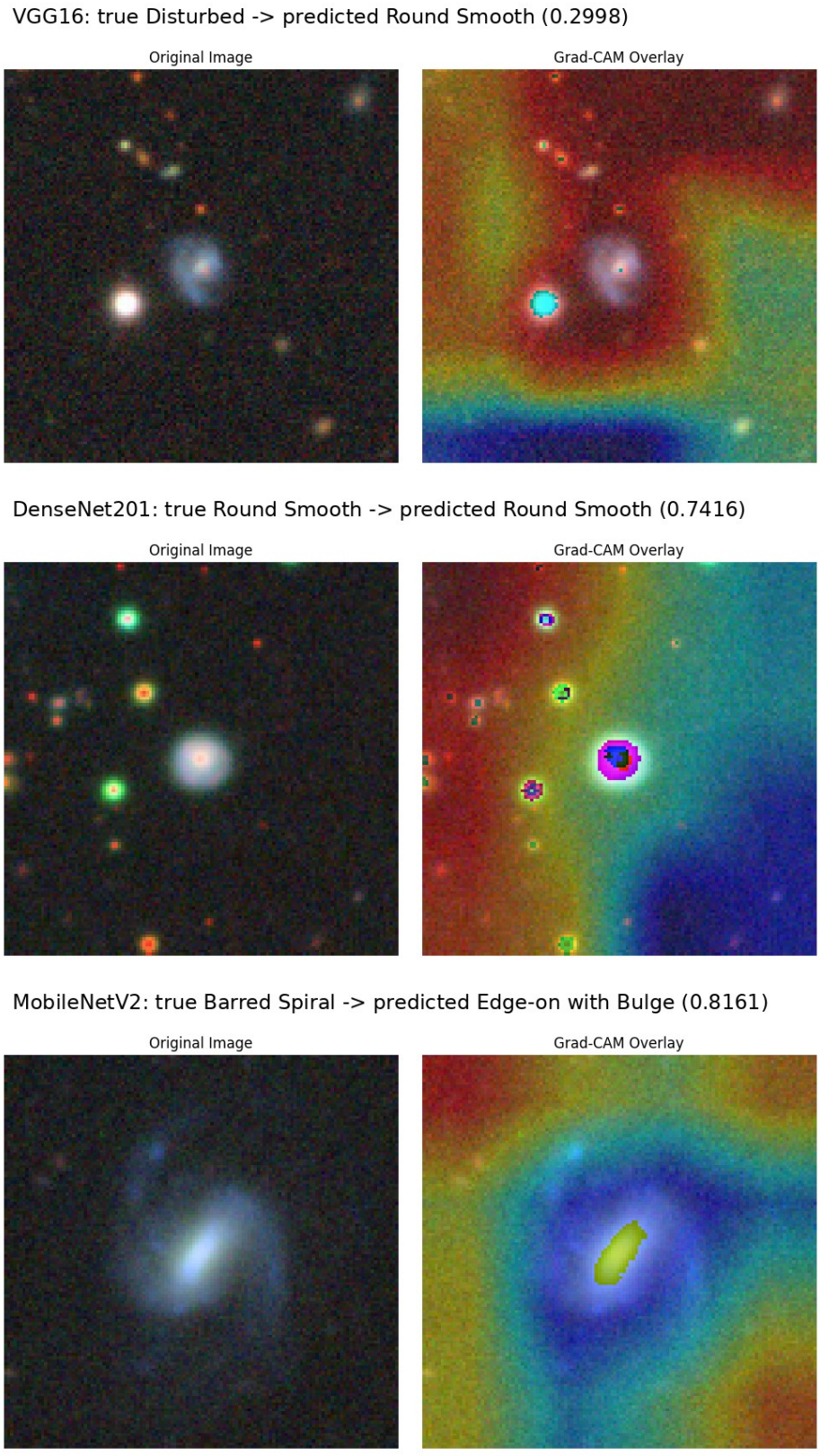

Figure 15. Representative Grad-CAM outputs produced by the notebook. Class names in the captions have been corrected to the official Galaxy10 taxonomy.

7.2 Interpretation of the displayed examples

- **VGG16 example.** A disturbed galaxy is predicted as Round Smooth with low confidence (0.2998). The broad activation pattern suggests that the model is not isolating a distinctive disturbance cue.
- **DenseNet201 example.** A Round Smooth galaxy is correctly predicted with 0.7416 confidence. The activation is concentrated near the bright central object, consistent with the morphology being dominated by global smoothness.
- **MobileNetV2 example.** A Barred Spiral galaxy is predicted as Edge-on with Bulge at high confidence (0.8161). The activation focuses on a compact central elongated region, which may explain the incorrect orientation-based interpretation.

7.3 Why the current Grad-CAM comparison is not fully controlled

> **Critical preprocessing mismatch**
>
> Training and test evaluation use standardized arrays (X_test_norm_std), but the Grad-CAM cells select X_test_norm, which is only divided by 255. The models therefore receive a different input distribution during explanation than during evaluation. The displayed prediction confidence and attention map may not represent the behavior of the evaluated model on its intended input.

- **Different images per model.** Each Grad-CAM cell draws a new random sample, so the models are not being compared on identical objects.
- **No fixed random seed.** The selected examples vary across runs.
- **Display and model tensors should be separated.** The model should receive the exact preprocessed tensor used in training, while a clean RGB copy should be retained only for visualization.
- **Heatmaps need quantitative checks.** A visually attractive overlay is not proof of causal reliance. Occlusion tests, deletion/insertion curves or sanity checks could strengthen the explanation.

8. Critical Assessment of the Implementation

8.1 What was done well

- **Training-only statistics.** Normalization statistics are computed from training data, avoiding direct validation/test leakage.
- **Stratified splits.** Rare classes are represented in all three partitions.
- **Consistent evaluation set.** All models are evaluated on the same 1,331 test images, and test_flow is not shuffled.
- **Multiple metrics.** The project reports precision, recall and F1 rather than relying solely on accuracy.
- **Model checkpoints and early stopping.** These callbacks limit unnecessary training and preserve the best validation-loss state.
- **Transfer-learning comparison.** The work clearly demonstrates that pretrained representations outperform the small custom CNN under the available resource constraints.

8.2 Main limitations and inconsistencies

Area	Observed issue	Recommended correction
Class semantics	Classes 6, 7 and 9 are mislabeled in the notebook.	Correct the mapping using galaxy10cls_lookup or the official taxonomy before any analysis.
Sampling reproducibility	The 50% random sample has no seed and is not stratified.	Set a NumPy seed or sample stratified indices; save the selected indices.
Pretrained preprocessing	All backbones receive the same generic standardization.	Use the matching preprocess_input function for each backbone or explicitly justify a shared transformation.
Augmentation scope	Only the rarest class is augmented, offline.	Compare class weights, balanced sampling and online augmentation across all classes.
Custom CNN description	Slides say ReLU; code uses tanh.	Ensure architecture diagrams and report text are generated from the code.
ResNet head	Flatten creates 327,690 trainable parameters and overfits.	Use GlobalAveragePooling2D, dropout and possibly L2 regularization.

Area	Observed issue	Recommended correction
Metric reporting	Final-epoch train/val metrics are mixed with restored-best test metrics.	Report the selected epoch and evaluate train/val/test with the same restored checkpoint.
Presentation table	Custom CNN macro F1 is shown as 0.52 rather than 0.22.	Populate tables directly from saved metric objects.
Confusion matrices	Only VGG16 and DenseNet201 matrices are saved.	Save y_true, y_pred and a confusion matrix for every model.
Grad-CAM inputs	Non-standardized images are fed to models trained on standardized images.	Use identical model preprocessing and the same test indices for all models.
Uncertainty	One random run is reported.	Repeat training with several seeds and report mean plus standard deviation or confidence intervals.
Fine-tuning	All pretrained layers remain frozen.	After head convergence, unfreeze selected upper blocks with a low learning rate.

Table 7. Technical audit of the submitted implementation.

8.3 Fairness of the architecture comparison

The models share the same sampled data, labels, batch size, optimizer family and stopping rule, which provides a useful baseline comparison. Nevertheless, the experiment is not a perfectly controlled architecture benchmark. ResNet50 uses a much larger Flatten head, each pretrained architecture normally expects different input preprocessing, and early stopping selects different epochs. No hyperparameter search is performed separately for each model. The results should therefore be interpreted as the outcome of this specific pipeline rather than a definitive ranking of the architectures.

9. Recommended Improved Pipeline

1. Correct the class mapping immediately and derive names programmatically from the dataset utility rather than maintaining a manual list.

2. Fix all random seeds (Python, NumPy and TensorFlow), enable deterministic operations where practical, and save the exact sampled indices.

3. Prefer the full 17,736-image dataset when hardware permits. If subsampling is necessary, use a stratified sample and document the seed.

4. Create one immutable stratified train/validation/test split before augmentation. Never augment validation or test images.

5. Keep raw RGB images for display, but prepare a separate model input for each architecture using its official preprocess_input routine.

6. Apply moderate online augmentation to the training set. Evaluate class weights, balanced minibatches or focal loss instead of synthesizing only one class.

7. Use GlobalAveragePooling2D for every backbone so that classifier capacity is comparable. Add dropout and L2 regularization only after validation-based tuning.

8. Train the classification head first. Then unfreeze the upper backbone blocks and fine-tune at a low learning rate such as 1e-5, with ReduceLROnPlateau or a scheduled learning rate.

9. Use the restored best checkpoint to recompute training, validation and test metrics. Record the selected epoch explicitly.

10. Save probabilities, predicted indices, confusion matrices and classification reports for every model. Include macro F1 as a primary selection metric.

11. Repeat each experiment with multiple seeds and report mean, standard deviation and class-level confidence intervals.

12. For Grad-CAM, use the same test images and the exact matching model preprocessing across architectures; retain unprocessed copies only for overlays.

13. Consider hierarchical or ordinal modeling because some categories lie on morphological continua, such as Round Smooth versus In-between Round Smooth and tight versus loose spirals.

Suggested model-selection criterion

Select the final system using validation macro F1, with test accuracy and weighted F1 reported as secondary metrics. This discourages a model from achieving a good headline score while ignoring rare classes.

10. Conclusions

The project successfully implements an end-to-end deep-learning workflow for ten-class galaxy morphology classification under computational constraints. Its central empirical result is clear: pretrained convolutional features transfer substantially better than the custom CNN trained from scratch. DenseNet201 gives the best overall saved result, reaching 57.48% test accuracy and 0.56 weighted F1, while ResNet50 is close but exhibits a large training-validation gap. VGG16 is a reliable intermediate baseline, and MobileNetV2 trades accuracy for efficiency.

The class-level analysis is more informative than the aggregate accuracy. Smooth and edge-on categories are recognized relatively well, whereas disturbed and cigar-shaped galaxies remain difficult. Many mistakes follow physically plausible visual similarities, including confusion between round and in-between smooth galaxies and among spiral subclasses.

At the same time, the technical audit identifies issues that materially affect interpretation: incorrect class names, non-reproducible initial sampling, absent architecture-specific preprocessing, inconsistent final-versus-restored metric reporting, and a Grad-CAM input mismatch. These issues do not erase the experimental result, but they limit the strength of the scientific conclusions. Correcting them, adding controlled fine-tuning and repeating runs with fixed seeds would turn the current educational experiment into a much more rigorous benchmark.

Final takeaway

Transfer learning is the decisive advantage in this experiment, but reliable scientific machine learning requires the data semantics, preprocessing, checkpoint selection and explanation pipeline to be as carefully controlled as the neural-network architecture itself.

References

[1] astroNN documentation. “Galaxy10 DECaLS Dataset.” Official dataset description and class taxonomy, accessed for verification of the label order.

[2] Simonyan, K., and Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. International Conference on Learning Representations.

[3] He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep Residual Learning for Image Recognition. Proceedings of CVPR, 770-778.

[4] Huang, G., Liu, Z., van der Maaten, L., and Weinberger, K. Q. (2017). Densely Connected Convolutional Networks. Proceedings of CVPR, 4700-4708.

[5] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. Proceedings of CVPR, 4510-4520.

[6] Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. Proceedings of ICCV, 618-626.

[7] Tram, L., Ibrahim, N., Nguyen, T., Noiplab, T., Kim, J., and Bhatti, D. (2024). Enhanced Comparative Analysis of Pretrained and Custom Deep Convolutional Neural Networks for Galaxy Morphology Classification. IEEE ICKII 2024.

[8] Wang, C. (2023). Galaxy morphology classification with densenet. Journal of Physics: Conference Series, 2580, 012064. DOI: 10.1088/1742-6596/2580/1/012064.

[9] Project source material: FDL_teamProci.ipynb and the submitted A.A. 2024/2025 presentation.

Official Galaxy10 DECaLS documentation: astronn.readthedocs.io/en/stable/galaxy10.html

Appendix A. Full Class-Level Results

The following tables transcribe the notebook classification reports while replacing the incorrect textual labels with the official class names. Values are rounded exactly as in the saved output. Support is identical for all models because they use the same test set.

A.1 Custom CNN

Index	Class	Precision	Recall	F1	Support
0	Disturbed	0.00	0.00	0.00	78
1	Merging	0.31	0.25	0.28	143
2	Round Smooth	0.35	0.33	0.34	197
3	In-between Round Smooth	0.35	0.38	0.36	151
4	Cigar-shaped Smooth	0.00	0.00	0.00	23
5	Barred Spiral	0.32	0.17	0.23	150
6	Unbarred Tight Spiral	0.29	0.17	0.22	145
7	Unbarred Loose Spiral	0.29	0.73	0.42	200
8	Edge-on without Bulge	0.00	0.00	0.00	106
9	Edge-on with Bulge	0.30	0.43	0.36	138
-	Macro average	-	-	0.22	1331
-	Weighted average	-	-	0.27	1331
-	Accuracy	-	-	0.3125	1331

A.2 VGG16

Index	Class	Precision	Recall	F1	Support
0	Disturbed	0.54	0.09	0.15	78
1	Merging	0.42	0.47	0.45	143
2	Round Smooth	0.54	0.85	0.66	197
3	In-between Round Smooth	0.61	0.44	0.51	151
4	Cigar-shaped Smooth	0.50	0.04	0.08	23
5	Barred Spiral	0.57	0.39	0.46	150
6	Unbarred Tight Spiral	0.46	0.46	0.46	145
7	Unbarred Loose Spiral	0.48	0.48	0.48	200
8	Edge-on without Bulge	0.84	0.78	0.81	106
9	Edge-on with Bulge	0.62	0.83	0.71	138
-	Macro average	-	-	0.48	1331
-	Weighted average	-	-	0.53	1331
-	Accuracy	-	-	0.5477	1331

A.3 DenseNet201

Index	Class	Precision	Recall	F1	Support
0	Disturbed	0.32	0.19	0.24	78
1	Merging	0.49	0.45	0.47	143
2	Round Smooth	0.58	0.77	0.66	197
3	In-between Round Smooth	0.53	0.56	0.54	151
4	Cigar-shaped Smooth	0.33	0.13	0.19	23
5	Barred Spiral	0.52	0.61	0.56	150
6	Unbarred Tight Spiral	0.54	0.51	0.53	145
7	Unbarred Loose Spiral	0.58	0.43	0.50	200
8	Edge-on without Bulge	0.87	0.75	0.81	106
9	Edge-on with Bulge	0.68	0.82	0.75	138
-	Macro average	-	-	0.52	1331
-	Weighted average	-	-	0.56	1331
-	Accuracy	-	-	0.5748	1331

A.4 ResNet50

Index	Class	Precision	Recall	F1	Support
0	Disturbed	0.40	0.10	0.16	78

Index	Class	Precision	Recall	F1	Support
1	Merging	0.58	0.60	0.59	143
2	Round Smooth	0.59	0.78	0.67	197
3	In-between Round Smooth	0.52	0.75	0.62	151
4	Cigar-shaped Smooth	0.24	0.43	0.31	23
5	Barred Spiral	0.56	0.43	0.48	150
6	Unbarred Tight Spiral	0.59	0.36	0.45	145
7	Unbarred Loose Spiral	0.44	0.48	0.46	200
8	Edge-on without Bulge	0.73	0.92	0.81	106
9	Edge-on with Bulge	0.83	0.53	0.65	138
-	Macro average	-	-	0.52	1331
-	Weighted average	-	-	0.55	1331
-	Accuracy	-	-	0.5665	1331

A.5 MobileNetV2

Index	Class	Precision	Recall	F1	Support
0	Disturbed	0.39	0.09	0.15	78
1	Merging	0.34	0.38	0.36	143
2	Round Smooth	0.49	0.79	0.60	197
3	In-between Round Smooth	0.60	0.26	0.37	151
4	Cigar-shaped Smooth	0.60	0.26	0.36	23
5	Barred Spiral	0.49	0.24	0.32	150
6	Unbarred Tight Spiral	0.41	0.42	0.42	145
7	Unbarred Loose Spiral	0.40	0.55	0.46	200
8	Edge-on without Bulge	0.75	0.80	0.77	106
9	Edge-on with Bulge	0.65	0.70	0.67	138
-	Macro average	-	-	0.45	1331
-	Weighted average	-	-	0.47	1331
-	Accuracy	-	-	0.4884	1331

Appendix B. Reproducibility and Artifact Audit

B.1 Saved artifacts available in the notebook

Artifact	Status	Consequence
Dataset distribution	Available	Before/after minority augmentation
Training curves	Available for all five models	Accuracy and loss
Classification reports	Available for all five models	Rounded precision, recall, F1 and support
Confusion matrices	Available only for VGG16 and DenseNet201	Exact matrices included in this report
Model checkpoints	Referenced but not included with the submitted files	Cannot reproduce predictions without rerunning
Predicted probabilities or y_pred arrays	Not persisted in the notebook file	Prevents reconstruction of missing matrices
Grad-CAM figures	Available for VGG16, DenseNet201 and MobileNetV2	Three random examples per model
Random sample indices	Not saved	Initial 50% sample is not reproducible

B.2 Minimum reproducibility checklist

- ☐ Pin package versions and record Python, TensorFlow, CUDA and cuDNN versions.
- ☐ Set and log Python, NumPy and TensorFlow random seeds.
- ☐ Save sampled and split indices.
- ☐ Save training-set channel means and standard deviations, or the exact preprocess_input choice.
- ☐ Save model architecture, best checkpoint, optimizer settings and selected epoch.
- ☐ Save y_true, y_pred and probability vectors for every evaluated model.
- ☐ Generate all report tables and figures from machine-readable result files rather than manual transcription.
- ☐ Use the official class lookup function to prevent label drift.